

Operating Systems: Assignment-8

Name: Suraj Shah

Admission Number: I24AI012

Q 1. Write a C program to create a single thread and print messages from both main and child thread.

Code:

```
#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    printf("Hello from child thread!\n");
    return NULL;
}

int main() {
    pthread_t thread_id;

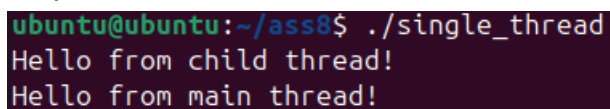
    if (pthread_create(&thread_id, NULL, thread_function, NULL) != 0) {
        printf("Error creating thread\n");
        return 1;
    }

    printf("Hello from main thread!\n");

    pthread_join(thread_id, NULL);

    return 0;
}
```

Output:



```
ubuntu@ubuntu:~/ass8$ ./single_thread
Hello from child thread!
Hello from main thread!
```

Q 2. Write a C program to create two threads to print odd and even numbers.

Code:

```
#include <stdio.h>
#include <pthread.h>

void* print_even(void* arg) {
    for(int i = 0; i <= 10; i++) {
        if(i % 2 == 0) {
            printf("Even: %d\n", i);
        }
    }
    return NULL;
}

void* print_odd(void* arg) {
    for(int i = 0; i <= 10; i++) {
        if(i % 2 != 0) {
            printf("Odd: %d\n", i);
        }
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, print_even, NULL);
    pthread_create(&t2, NULL, print_odd, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

Output:

```
ubuntu@ubuntu:~/ass8$ ./two_threads
Even: 0
Even: 2
Even: 4
Even: 6
Even: 8
Even: 10
Odd: 1
Odd: 3
Odd: 5
Odd: 7
Odd: 9
```

Q 3. Explain the use of `pthread_create()` and `pthread_join()`.

Answer:

`pthread_create()`

Purpose:

Used to create a new thread in a process.

Syntax:

```
int pthread_create(pthread_t *thread,  
                  const pthread_attr_t *attr,  
                  void *(*start_routine)(void *),  
                  void *arg);
```

Parameters:

- `thread` → Pointer to thread ID (used to identify the thread)
- `attr` → Thread attributes (usually NULL for default)
- `start_routine` → Function the thread will execute
- `arg` → Argument passed to the function

Return:

- 0 → Success
- Non-zero → Error

Example:

```
pthread_t t;  
pthread_create(&t, NULL, myFunction, NULL);  
This creates a new thread that runs myFunction.
```

`pthread_join()`

Purpose:

Used to wait for a thread to finish execution.

Syntax:

```
int pthread_join(pthread_t thread, void **retval);
```

Parameters:

- `thread` → Thread ID to wait for
- `retval` → Pointer to store return value (usually NULL)

Return:

- 0 → Success
- Non-zero → Error

Example:

```
pthread_join(t, NULL);  
Main thread waits until thread t completes.
```

Q 4. Write a multithreaded C program to find maximum and minimum in an array using two threads.

Code:

```
#include <stdio.h>
#include <pthread.h>

#define SIZE 10

int arr[SIZE];
int max, min;

void* find_max(void* arg) {
    max = arr[0];
    for(int i = 1; i < SIZE; i++) {
        if(arr[i] > max) {
            max = arr[i];
        }
    }
    printf("Maximum value: %d\n", max);
    return NULL;
}

void* find_min(void* arg) {
    min = arr[0];
    for(int i = 1; i < SIZE; i++) {
        if(arr[i] < min) {
            min = arr[i];
        }
    }
    printf("Minimum value: %d\n", min);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("Enter %d elements:\n", SIZE);
    for(int i = 0; i < SIZE; i++) {
        scanf("%d", &arr[i]);
    }
}
```

```
pthread_create(&t1, NULL, find_max, NULL);
pthread_create(&t2, NULL, find_min, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

return 0;
}
```

Output:

```
ubuntu@ubuntu:~/ass8$ ./min_max
Enter 10 elements:
5 12 3 9 45 2 18 7 30 1
Maximum value: 45
Minimum value: 1
```

Q 5. Write a program to compute factorial and sum of first N natural numbers using two threads.

Code:

```
#include <stdio.h>
#include <pthread.h>

int n;
long long factorial = 1;
int sum = 0;

void* calc_factorial(void* arg) {
    factorial = 1;
    for(int i = 1; i <= n; i++) {
        factorial *= i;
    }
    printf("Factorial of %d = %lld\n", n, factorial);
    return NULL;
}

void* calc_sum(void* arg) {
    sum = 0;
    for(int i = 1; i <= n; i++) {
        sum += i;
    }
    printf("Sum of first %d natural numbers = %d\n", n, sum);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("Enter a number: ");
    scanf("%d", &n);

    pthread_create(&t1, NULL, calc_factorial, NULL);
    pthread_create(&t2, NULL, calc_sum, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

}

Output:

```
ubuntu@ubuntu:~/ass8$ ./factorial_sum
Enter a number: 5
Factorial of 5 = 120
Sum of first 5 natural numbers = 15
```


Q 6. Explain the concept of race condition.

Answer:

A race condition occurs when two or more threads access shared data at the same time, and the final result depends on the order of execution of those threads.

Why it Happens

- Multiple threads share the same variable/resource
- No proper synchronization (like mutex/locks)
- Execution order is not controlled

Example

```
int counter = 0;
```

```
// Thread 1
```

```
counter++; // read → modify → write
```

```
// Thread 2
```

```
counter++; // read → modify → write
```

What should happen:

Final value = 2

What may happen:

Both threads read 0 at the same time → both write 1

Final value = 1

Step-by-Step Problem

1. Thread 1 reads counter = 0
2. Thread 2 reads counter = 0
3. Thread 1 updates → 1
4. Thread 2 updates → 1 (overwrites)

This is a race condition

Effects

- Incorrect results
- Data inconsistency
- Hard-to-debug errors
- Unpredictable program behavior

Q 7. Write a C program to demonstrate race condition in a shared counter.

Code:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void* increment(void* arg) {
    for(int i = 0; i < 100000; i++) {
        counter++; // Not synchronized → race condition
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

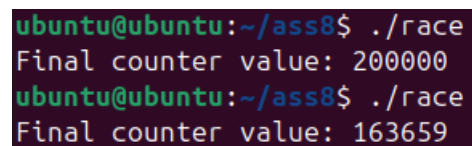
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter value: %d\n", counter);

    return 0;
}
```

Output:



```
ubuntu@ubuntu:~/ass8$ ./race
Final counter value: 200000
ubuntu@ubuntu:~/ass8$ ./race
Final counter value: 163659
```

Q 8. Modify the program using mutex to avoid race condition.

Code:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void* increment(void* arg) {
    for(int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter value: %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

Output:

```
ubuntu@ubuntu:~/ass8$ ./race_mutex
Final counter value: 200000
```

Q 9. Write a multithreaded program for matrix addition or matrix multiplication.

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

int **A, **B, **C;
int n;

void* add_row(void* arg) {
    int row = *(int*)arg;

    for(int j = 0; j < n; j++) {
        C[row][j] = A[row][j] + B[row][j];
    }

    return NULL;
}

int main() {
    printf("Enter size of matrix (n for nxn): ");
    scanf("%d", &n);

    A = (int**)malloc(n * sizeof(int*));
    B = (int**)malloc(n * sizeof(int*));
    C = (int**)malloc(n * sizeof(int*));

    for(int i = 0; i < n; i++) {
        A[i] = (int*)malloc(n * sizeof(int));
        B[i] = (int*)malloc(n * sizeof(int));
        C[i] = (int*)malloc(n * sizeof(int));
    }

    printf("Enter elements of Matrix A:\n");
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            scanf("%d", &A[i][j]);
        }
    }

    printf("Enter elements of Matrix B:\n");
```

```

for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        scanf("%d", &B[i][j]);
    }
}

pthread_t threads[n];
int row_index[n];

for(int i = 0; i < n; i++) {
    row_index[i] = i;
    pthread_create(&threads[i], NULL, add_row, &row_index[i]);
}

for(int i = 0; i < n; i++) {
    pthread_join(threads[i], NULL);
}

printf("\nResultant Matrix (A + B):\n");
for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        printf("%d ", C[i][j]);
    }
    printf("\n");
}

for(int i = 0; i < n; i++) {
    free(A[i]);
    free(B[i]);
    free(C[i]);
}
free(A);
free(B);
free(C);

return 0;
}

```

Output:

```
ubuntu@ubuntu:~/ass8$ ./matrix_addition
Enter size of matrix (n for nxn): 3
Enter elements of Matrix A:
1 2 3
4 5 6
7 8 9
Enter elements of Matrix B:
9 8 7
6 5 4
3 2 1

Resultant Matrix (A + B):
10 10 10
10 10 10
10 10 10
```

Q 10. Explain thread synchronization and critical section.

Answer:

Thread Synchronization

Thread synchronization is the technique used to control the execution of multiple threads so that they access shared resources safely and avoid errors like race conditions.

Why Synchronization is Needed

- Multiple threads share data (variables, files, memory)
- Without control → data corruption / inconsistent results
- Prevents race conditions

Common Synchronization Techniques

- Mutex (Lock) → Only one thread at a time
- Semaphore → Limited number of threads allowed
- Monitor → High-level synchronization mechanism

Critical Section

A critical section is the part of the program where shared resources are accessed or modified.

Example

```
counter++; // Critical section
```

Why?

- It modifies shared variable counter
- If multiple threads execute it together → incorrect result

Protecting Critical Section (Using Mutex)

```
pthread_mutex_lock(&lock); // Enter critical section
```

```
counter++;           // Critical section
```

```
pthread_mutex_unlock(&lock); // Exit critical section
```